L10. Arithmetic Logic Design (2)

Lecture 10

10.1 ALU (교재 5.7)

- ALU(arithmetic logic unit): 정수 이진수에 산술 또는 논리 연산을 수행하는 조합 네트워크.
  부동소수점 수를 다루는 FPU 와 대비됨.
- 1비트 slice 를 설계하고 cascade 해서 multi-bit 구조를 만듦. 성능을 제한하는 요인은 carry 의 전파.

참고: IEEE 754 부동소수점 비트 분해 (FPU 가 다루는 수 표현)

부호 s

지수 e (8비트, 편향 127)

가수 f (23비트)

필드비트해석부호 s1(−1)1 = −1지수 e10000001e = 129, E = e − 127 = 2가수 f10010000000000000000000암묵적 1을 붙여 1.1001_2 = 1.5625

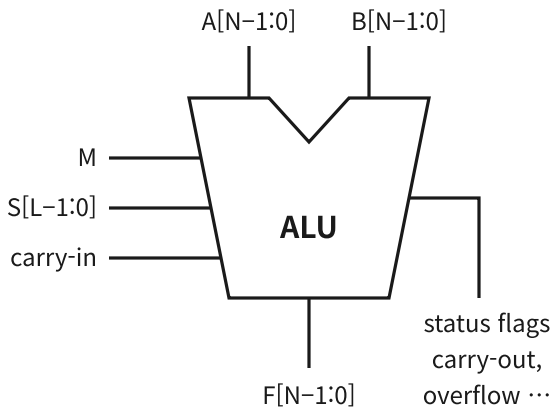
저장된 값(정확한 십진값) = −6.25

인접 표현 간격(ulp) = 2^{-21} = 0.000000476837158203125

16진수 0XC0C80000

입력값 -6.25은(는) 이 형식에서 정확히 표현된다.

십진 입력은 먼저 JavaScript Number(double)로 해석되며, half/single은 그 값에서 다시 반올림된다.



ALU 블록. M + S 가 OpCode, 입력 status 는 보통 carry-in 하나, 출력 status 는 flag.

1비트 slice 의 기능 표

M	C_0	S_0	S_1	F	동작
0 (논리)	–	0	0	A_i	A 를 출력으로
	–	1	0	NOT A_i	A 의 보수
	–	0	1	A_i XOR B_i	XOR
	–	1	1	A_i XNOR B_i	XNOR
1 (산술)	0	0	0	A	A 전달
	0	1	0	NOT A	A 의 보수 전달
	0	0	1	A + B	합
	0	1	1	(NOT A) + B	B 와 A 의 보수의 합
1 (산술)	1	0	0	A + 1	increment A
	1	1	0	(NOT A) + 1	A 의 2의 보수
	1	0	1	A + B + 1	합의 increment
	1	1	1	B + (NOT A) + 1	B minus A

carry-in = 1, S_0S_1 = 11 이면 F = B + (NOT A) + 1 = B − A (B + A 의 2의 보수).

이 기능 표는 강의 원본 슬라이드 미확보로 원본 대조 전. 검증된 것은 $X_1 = S_0 \oplus A_i$ 구조라는 가정 아래의 논리 자기정합성뿐.

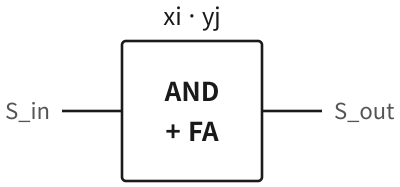
10.2 조합 곱셈기 (교재 5.8)

조합 곱셈기 (Combinational multiplier)

- 곱셈기를 조합 논리로 구현. 예: 4비트 × 4비트. 부분곱은 A_iB_j 의 AND.

				A_3	**A_2**	**A_1**	**A_0**
				A_3B_0	A_2B_0	A_1B_0	A_0B_0
			A_3B_1	A_2B_1	A_1B_1	A_0B_1	
		A_3B_2	A_2B_2	A_1B_2	A_0B_2		
	A_3B_3	A_2B_3	A_1B_3	A_0B_3			
S_7	S_6	S_5	S_4	S_3	S_2	S_1	S_0

- 부분곱 16개 = AND 게이트 16개. 덧셈은 가산기 12개(full adder 는 각 6 게이트, 72 게이트). 합쳐서 88 게이트. 이 예의 full adder를 6게이트로 구현한 경우의 수치이며, 일반적인 고정값은 아님.
- 다르게 그리면 4 × 4 배열의 빌딩 블록. 블록 하나 = full adder + AND 게이트.



셀 하나: 부분곱 x·y 를 AND 로 만들고 FA 가 위쪽 sum 과 옆 carry 에 더한다.

10.3 여러 수의 덧셈과 carry save adder

- k개의 N비트 수를 더하는 경우. 예: 0001 + 0111 + 1101 + 0010 = 10111 (1 + 7 + 13 + 2 = 23). 단순하게는 N비트 가산기를 k−1 개 쓰면 되지만 크고 느림.

Carry save addition

- full adder 하나는 입력 3개를 받아 출력 2개(sum, carry)를 냄.
- carry 는 sum 의 2배 가중치.
- full adder N개를 병렬로 둔 것이 carry save adder(CSA)이며 N개의 sum 과 N개의 carry-out 이 나옴.
- X + Y + Z = S + 2C.

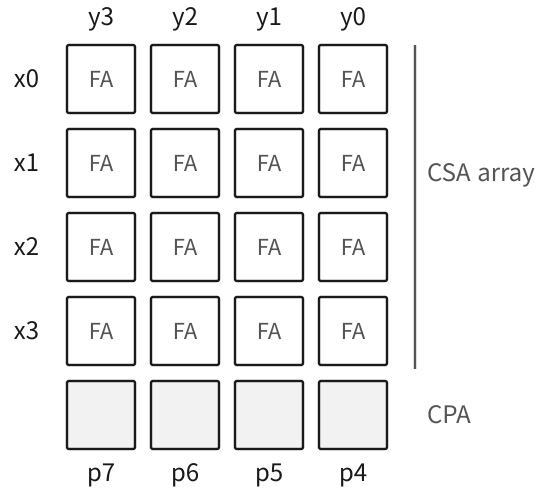
CSA 로 0001 + 0111 + 1101 + 0010

단계	입력	S	C (2C)
4-bit CSA	X = 0001, Y = 0111, Z = 1101	1011	0101 → 01010
5-bit CSA	X = 01010 (2C), Y = 01011 (S), Z = 00010	00011	01010 → 10100
마지막 덧셈	00011 + 10100	10111	

- CSA 는 k − 2 단(수 하나씩 순서대로 넣을 때).
- 중간 결과를 carry-save 형태(S 와 2C 두 수)로 두고, 마지막 단에서 두 수를 더해 실제 합을 구함.
- carry 전파는 마지막 덧셈 단계로 미룸.
- 결과 10111 = 23.

10.4 CSA 를 이용한 부분곱 덧셈

- 곱셈기의 임계 경로는 부분곱 덧셈 쪽.
- CSA 배열 곱셈기.
- 위쪽 CSA array 가 부분곱을 더하고, 맨 아래 한 행의 CPA(carry propagate adder)가 마지막 carry 를 전파.
- critical path는 이 구조에서 CSA array 를 대각선으로 지나 CPA 로 이어짐.



4×4 array multiplier 구조. 위 4행은 CSA array, 맨 아래 한 행이 CPA 다. 곱의 하위 비트 p3~p0 는 각 행의 가장 오른쪽 셀에서 나온다.

Wallace tree

- 부분곱 덧셈이 곱셈기의 임계 경로. array 구조에서도 여러 CSA가 동시에 동작함. 하지만 앞 단계의 결과가 다음 단계로 차례로 전파되어 지연이 누적됨.
- Wallace tree: (3, 2) CSA(입력 3개 → sum, carry 2개)로 부분곱을 병렬로 압축. 같은 단계에서 여러 CSA 가 동시에 동작.
- 압축이 끝나면 남은 두 수(carry-save 형태)를 CPA 로 더해 실제 곱을 얻음.

구조	덧셈 방식	필요한 CSA 단 수
Array (column addition)	연결된 CSA 단계들을 따라 지연 누적	$N - 2$
Wallace tree	병렬 — 부분곱을 동시에 압축	N개 부분곱 행이 단계마다 약 2/3로 줄어든다고 보면 대략 $\log_{3/2}(N/2)$. 정확한 단계 수는 배치에 따라 다름